

Lab 8 & Lab 7 - RDP for C Grammer

Token.h:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define BUFFER_SIZE 100
#define TOKEN_MAX_LEN 100

struct token {
    char token_name[TOKEN_MAX_LEN];
    int index;
    unsigned int row, col;
    char type[TOKEN_MAX_LEN];
};

void initLexer(const char *path);
int getNextToken(struct token *tok);

int isArithOp(const char *str);
int isRelOp(const char *str);
int isLogOp(const char *str);
int isNum(const char *str);
int isStringLit(const char *str);
int isKey(const char *str);
int isIden(const char *str);
int isSpecSym(char c);
void idTokenType(struct token *item);

static const char *reserved_words[] = {
    "int", "float", "char", "if", "else",
    "for", "while", "return", "void", "switch"
};

#define TOTAL_RESERVED (sizeof(reserved_words) / sizeof(reserved_words[0]))

static FILE *input_stream = NULL;
static unsigned int current_row = 1, current_col = 0;
static int lookahead_char = 0;

void initLexer(const char *path) {
    input_stream = fopen(path, "r");
```

```

    if (!input_stream) {
        perror("File Open Error");
        exit(1);
    }
    lookahead_char = fgetc(input_stream);
    current_row = 1;
    current_col = 1;
}

static int advance_cursor() {
    if (lookahead_char == EOF)
        return EOF;
    int previous = lookahead_char;
    lookahead_char = fgetc(input_stream);
    if (previous == '\n') {
        current_row++;
        current_col = 1;
    } else {
        current_col++;
    }
    return previous;
}

static void clear_whitespace() {
    while (isspace(lookahead_char))
        advance_cursor();
}

static void ignore_line() {
    while (lookahead_char != '\n' && lookahead_char != EOF)
        advance_cursor();
}

static void ignore_block_comment() {
    advance_cursor();
    while (lookahead_char != EOF) {
        if (lookahead_char == '*') {
            advance_cursor();
            if (lookahead_char == '/') {
                advance_cursor();
                break;
            }
        }
        else {
            advance_cursor();
        }
    }
}

```

```

    }
}

static void capture_word(char *buffer) {
    int idx = 0;
    while (isalnum(lookahead_char) || lookahead_char == '_') {
        if (idx < BUFFER_SIZE - 1)
            buffer[idx++] = advance_cursor();
        else
            advance_cursor();
    }
    buffer[idx] = '\0';
}

static void capture_operator(char *buffer) {
    int idx = 0;
    buffer[idx++] = advance_cursor();
    char next = lookahead_char;
    char first = buffer[0];
    if ((first == '+' && next == '+') ||
        (first == '-' && next == '-') ||
        (first == '=' && next == '=') ||
        (first == '!' && next == '=') ||
        (first == '<' && next == '=') ||
        (first == '>' && next == '=') ||
        (first == '&' && next == '&') ||
        (first == '|' && next == '|')) {
        buffer[idx++] = advance_cursor();
    }
    buffer[idx] = '\0';
}

int getNextToken(struct token *tok) {
    char text[BUFFER_SIZE];
    while (lookahead_char != EOF) {
        clear_whitespace();
        int active_char = lookahead_char;
        if (active_char == EOF)
            return 0;

        if (active_char == '#') {
            ignore_line();
            if (lookahead_char == '\n')
                advance_cursor();
            continue;
        }
    }
}

```

```

}

if (active_char == '/') {
    advance_cursor();
    if (lookahead_char == '/') {
        ignore_line();
        continue;
    } else if (lookahead_char == '*') {
        ignore_block_comment();
        continue;
    } else {
        text[0] = '/';
        text[1] = '\0';
        tok->row = current_row;
        tok->col = current_col - 1;
        strcpy(tok->token_name, text);
        idTokenType(tok);
        return 1;
    }
}

if (strchr("+-*%=<>!&|", active_char)) {
    tok->row = current_row;
    tok->col = current_col;
    capture_operator(text);
    strcpy(tok->token_name, text);
    idTokenType(tok);
    return 1;
}

if (isSpecSym(active_char)) {
    tok->row = current_row;
    tok->col = current_col;
    text[0] = advance_cursor();
    text[1] = '\0';
    strcpy(tok->token_name, text);
    idTokenType(tok);
    return 1;
}

if (isalnum(active_char) || active_char == '_') {
    tok->row = current_row;
    tok->col = current_col;
    capture_word(text);
    strcpy(tok->token_name, text);
    idTokenType(tok);
    return 1;
}

```

```

        advance_cursor();
    }
    return 0;
}

int isArithOp(const char *str) {
    return (!strcmp(str, "+") || !strcmp(str, "-") || !strcmp(str, "*") ||
            !strcmp(str, "/") || !strcmp(str, "%") || !strcmp(str, "++") ||
            !strcmp(str, "--"));
}

int isRelOp(const char *str) {
    return (!strcmp(str, "<") || !strcmp(str, ">") || !strcmp(str, "<=") ||
            !strcmp(str, ">=") || !strcmp(str, "==") || !strcmp(str, "!=") ||
            !strcmp(str, "="));
}

int isLogOp(const char *str) {
    return (!strcmp(str, "&&") || !strcmp(str, "||") || !strcmp(str, "!"));
}

int isNum(const char *str) {
    int i = 0, points = 0;
    if (str[i] == '+' || str[i] == '-') i++;
    for (; str[i]; i++) {
        if (str[i] == '.') {
            points++;
            if (points > 1) return 0;
        } else if (!isdigit(str[i])) return 0;
    }
    return (i > 0);
}

int isStringLit(const char *str) {
    size_t len = strlen(str);
    return (len >= 2 && str[0] == '"' && str[len - 1] == '"');
}

int isKey(const char *str) {
    for (int i = 0; i < TOTAL_RESERVED; i++)
        if (strcmp(str, reserved_words[i]) == 0) return 1;
    return 0;
}

int isIden(const char *str) {

```

```

    if (!isalpha(str[0]) && str[0] != '_') return 0;
    for (int i = 1; str[i]; i++)
        if (!isalnum(str[i]) && str[i] != '_') return 0;
    return !isKey(str);
}

int isSpecSym(char c) {
    return (c == ';' || c == ',' ||
            c == '(' || c == ')' ||
            c == '{' || c == '}' ||
            c == '[' || c == ']' ||
            c == ':' || c == '?');
}

void idTokenType(struct token *item) {
    const char *name = item->token_name;
    if (isKey(name)) strcpy(item->type, "kw");
    else if (isArithOp(name)) strcpy(item->type, "arithop");
    else if (isRelOp(name)) strcpy(item->type, "relop");
    else if (isLogOp(name)) strcpy(item->type, "logop");
    else if (isNum(name)) strcpy(item->type, "num");
    else if (isStringLit(name)) strcpy(item->type, "literal");
    else if (isIden(name)) strcpy(item->type, "id");
    else strcpy(item->type, "specsym");
}

```

G71_parser.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "token.h"

void initLexer(const char *filename);
int getNextToken(struct token *t);

struct token lookahead;
int error_flag = 0;

typedef struct {
    char token_name[100];
    char type[50];
    unsigned int row;
    unsigned int col;
} TokenRecord;

```

```

TokenRecord token_list[1000];
int token_count = 0;

void recordToken() {
    if (token_count < 1000) {
        strcpy(token_list[token_count].token_name, lookahead.token_name);
        strcpy(token_list[token_count].type, lookahead.type);
        token_list[token_count].row = lookahead.row;
        token_list[token_count].col = lookahead.col;
        token_count++;
    }
}

void syntaxError(const char *expected) {
    printf("Syntax Error at [%d, %d]: Expected %s, but found %s\n",
        lookahead.row, lookahead.col, expected, lookahead.token_name);
    error_flag = 1;
}

void match(const char *expected) {
    if (strcmp(lookahead.token_name, expected) == 0 ||
        strcmp(lookahead.type, expected) == 0) {
        recordToken();
        if (!getNextToken(&lookahead)) {
            strcpy(lookahead.token_name, "EOF");
        }
    } else {
        syntaxError(expected);
    }
}

void printTokenList() {
    printf("\n--- Processed Tokens ---\n");
    for (int i = 0; i < token_count; i++) {
        printf("%d. %s (%s) Line: %d Col: %d\n",
            i + 1, token_list[i].token_name, token_list[i].type,
            token_list[i].row, token_list[i].col);
    }
}

/* Grammar Functions */

void Program();
void declarations();
void data_type();

```

```

void identifier_list();
void statement_list();
void statement();
void assign_stat();
void expn();
void eprime();
void simple_expn();
void seprime();
void term();
void tprime();
void factor();
void decision_stat();
void dprime();

void Program() {
    if (strcmp(lookahead.token_name, "main") == 0)
        match("main");
    else
        syntaxError("main");

    match("(");
    match(")");
    match("{");

    declarations();
    statement_list();

    match("}");

    if (!error_flag)
        printf("Success: Parsing completed.\n");
}

void declarations() {
    if (strcmp(lookahead.token_name, "int") == 0 ||
        strcmp(lookahead.token_name, "float") == 0 ||
        strcmp(lookahead.token_name, "char") == 0) {
        data_type();
        identifier_list();
        match(";");
        declarations();
    }
}

void data_type() {

```



```

    if (strcmp(lookahead.token_name, "int") == 0) match("int");
    else if (strcmp(lookahead.token_name, "float") == 0) match("float");
    else if (strcmp(lookahead.token_name, "char") == 0) match("char");
    else syntaxError("data type");
}

void identifier_list() {
    if (strcmp(lookahead.type, "id") == 0) {
        match("id");
        if (strcmp(lookahead.token_name, "[") == 0) {
            match("[");
            if (strcmp(lookahead.type, "num") == 0) match("num");
            else syntaxError("number");
            match("]");
        }
        if (strcmp(lookahead.token_name, ",") == 0) {
            match(",");
            identifier_list();
        }
    } else {
        syntaxError("identifier");
    }
}

void statement_list() {
    if (strcmp(lookahead.type, "id") == 0 ||
        strcmp(lookahead.token_name, "if") == 0 ||
        strcmp(lookahead.token_name, "while") == 0 ||
        strcmp(lookahead.token_name, "for") == 0) {
        statement();
        statement_list();
    }
}

void statement() {
    if (strcmp(lookahead.token_name, "if") == 0) {
        decision_stat();
    } else if (strcmp(lookahead.token_name, "while") == 0 ||
               strcmp(lookahead.token_name, "for") == 0) {
        if (strcmp(lookahead.token_name, "while") == 0) match("while");
        else match("for");
        match("(");
        expn();
        match(")");
        match("{");
    }
}

```

```

        statement_list();
        match("}");
    } else if (strcmp(lookahead.type, "id") == 0) {
        assign_stat();
        match(";");
    } else {
        syntaxError("statement");
    }
}

void assign_stat() {
    match("id");
    match("=");
    expn();
}

void expn() {
    simple_expn();
    eprime();
}

void eprime() {
    if (strcmp(lookahead.type, "relop") == 0) {
        match("relop");
        simple_expn();
    }
}

void simple_expn() {
    term();
    seprime();
}

void seprime() {
    if (strcmp(lookahead.type, "arithop") == 0) {
        char op = lookahead.token_name[0];
        if (op == '+' || op == '-') {
            match("arithop");
            term();
            seprime();
        }
    }
}

void term() {

```

```

    factor();
    tprime();
}

void tprime() {
    if (strcmp(lookahead.type, "arithop") == 0) {
        char op = lookahead.token_name[0];
        if (op == '*' || op == '/' || op == '%') {
            match("arithop");
            factor();
            tprime();
        }
    }
}

void factor() {
    if (strcmp(lookahead.type, "id") == 0) match("id");
    else if (strcmp(lookahead.type, "num") == 0) match("num");
    else syntaxError("id or num");
}

void decision_stat() {
    match("if");
    match("(");
    expn();
    match(")");
    match("{");
    statement_list();
    match("}");
    dprime();
}

void dprime() {
    if (strcmp(lookahead.token_name, "else") == 0) {
        match("else");
        match("{");
        statement_list();
        match("}");
    }
}

int main() {
    initLexer("input.c");

    if (!getNextToken(&lookahead)) {

```

```
        printf("Empty file.\n");  
        return 0;  
    }  
  
    Program();  
    printTokenList();  
  
    if (error_flag) {  
        printf("Parsing failed.\n");  
        return 1;  
    }  
  
    return 0;  
}
```

Output:

Error:

```
}6CSEC1@db1-23:~/Documents/CD_230905152/L8$ gcc g71_parser.c -o abc
6CSEC1@db1-23:~/Documents/CD_230905152/L8$ ./abc
Syntax Error at [5, 5]: Expected =, but found b
```

--- Processed Tokens ---

1.	main	(id)	Line: 2	Col: 1
2.	(	(specsym)	Line: 2	Col: 5
3.	)	(specsym)	Line: 2	Col: 6
4.	{	(specsym)	Line: 2	Col: 7
5.	int	(kw)	Line: 3	Col: 5
6.	a	(id)	Line: 3	Col: 9
7.	,	(specsym)	Line: 3	Col: 10
8.	b	(id)	Line: 3	Col: 11
9.	,	(specsym)	Line: 3	Col: 12
10.	c	(id)	Line: 3	Col: 13
11.	;	(specsym)	Line: 3	Col: 14
12.	qwerty	(id)	Line: 3	Col: 15
13.	b	(id)	Line: 5	Col: 5
14.	=	(relop)	Line: 5	Col: 6
15.	10	(num)	Line: 5	Col: 7
16.	;	(specsym)	Line: 5	Col: 9
17.	}	(specsym)	Line: 6	Col: 1

Parsing failed.

```
6CSEC1@db1-23:~/Documents/CD_230905152/L8$ cat input.c
#include<stdio.h>
main(){
    int a,b,c;qwerty
    //Hi World
    b=10;
```

Correct:

```
6CSEC1@db1-23:~/Documents/CD_230905152/L8$ gcc g71_parser.c -o abc
```

```
6CSEC1@db1-23:~/Documents/CD_230905152/L8$ ./abc
```

Success: Parsing completed.

--- Processed Tokens ---

1.	main	(id)	Line: 2	Col: 1
2.	(	(specsymb)	Line: 2	Col: 5
3.	)	(specsymb)	Line: 2	Col: 6
4.	{	(specsymb)	Line: 2	Col: 7
5.	int	(kw)	Line: 3	Col: 5
6.	a	(id)	Line: 3	Col: 9
7.	,	(specsymb)	Line: 3	Col: 10
8.	b	(id)	Line: 3	Col: 11
9.	,	(specsymb)	Line: 3	Col: 12
10.	c	(id)	Line: 3	Col: 13
11.	;	(specsymb)	Line: 3	Col: 14
12.	b	(id)	Line: 5	Col: 5
13.	=	(relop)	Line: 5	Col: 6
14.	10	(num)	Line: 5	Col: 7
15.	;	(specsymb)	Line: 5	Col: 9
16.	}	(specsymb)	Line: 6	Col: 1

```
6CSEC1@db1-23:~/Documents/CD_230905152/L8$ cat input.c
```

```
#include<stdio.h>
```

```
main(){
```

```
    int a,b,c;
```

```
    //Hi World
```

```
    b=10;
```